

Design Document – PS9: City Surveillance Agent

BITS Pilani WILP | AIMLCZG557/AECLZG557 | Assignment 1 Group ID: G121 |
Submission Date: June 8, 2026

1. Problem Description

The City of Chennai has several tourist landmarks connected by lanes (roads). A surveillance drone must monitor all lanes by traversing each lane **exactly once**. Landmarks (nodes) may be visited more than once. The drone operates on a battery, making it essential to find an efficient path.

This is modelled as an **Edge Coverage Problem** on an undirected weighted graph, solved using **Greedy Best First Search (GBFS)**.

2. PEAS Description

Component	Description
Performance Measure	Cover all lanes exactly once; minimize total distance travelled; maximize battery efficiency
Environment	Chennai city map with 6 tourist landmarks as nodes and 7 lanes as edges; static, fully observable, deterministic
Actuators	Drone propellers (movement), camera (surveillance), battery management system, navigation controller
Sensors	GPS (position tracking), camera (lane monitoring), battery sensor, edge traversal tracker

3. Environmental Characteristics

Property	Value	Justification
----------	-------	---------------

Observable	Fully Observable	Drone has complete city map before starting
Agents	Single Agent	Only one drone operates
Deterministic	Yes	Moving from A to B always produces same result
Sequential	Yes	Each move depends on previously covered lanes
Static	Yes	City map does not change during surveillance
Discrete	Yes	Finite set of landmarks and lanes
Known	Yes	Complete map available to agent beforehand

4. Graph Representation

The city map is represented as an **undirected weighted graph** using an **adjacency list**.

Nodes (Landmarks): Marina Beach, Mahabalipuram, Vandalur Zoo, Kovalam Beach, Government Museum, Muttukadu

Edges (Lanes with distances in km):

From	To	Distance
Marina Beach	Mahabalipuram	60 km
Marina Beach	Government Museum	3 km
Mahabalipuram	Vandalur Zoo	55 km
Mahabalipuram	Kovalam Beach	15 km
Mahabalipuram	Government Museum	58 km
Vandalur Zoo	Kovalam Beach	45 km
Kovalam Beach	Muttukadu	10 km

Why Adjacency List?

- Efficient for sparse graphs (few edges per node)
- $O(1)$ neighbor lookup

- Memory efficient: $O(V + E)$ space
-

5. Heuristic Function

Formula: $h(n) = -(\text{number of unvisited edges at node } n)$

Rationale:

- The goal is to cover all lanes. Nodes with more unvisited edges offer more surveillance opportunities.
- Negative value is used because Python's `heapq` is a min-heap — negating converts "maximize unvisited edges" into "minimize heuristic value."
- This is an **informed heuristic** that directly measures remaining work at each node.

Example:

- Node A has 3 unvisited edges $\rightarrow h(A) = -3 \rightarrow$ **High Priority**
 - Node B has 1 unvisited edge $\rightarrow h(B) = -1 \rightarrow$ **Low Priority**
 - GBFS selects Node A first.
-

6. Algorithm: Greedy Best First Search (GBFS)

6.1 Data Structures Used

Structure	Purpose
<code>CityGraph</code> (Adjacency List)	Store city map
<code>PriorityQueue</code> (Min-Heap)	Select best next node using heuristic
<code>set</code> (visited_edges)	Track covered lanes
<code>list</code> (path)	Record surveillance route
<code>deque</code> (BFS queue)	Backtracking to find unvisited lanes

6.2 Algorithm Steps

1. Read starting node from inputPS9.txt

2. Initialise: visited_edges = {}, path = [start]
3. WHILE not all edges covered:
 - a. Push all unvisited neighbors to PriorityQueue with heuristic $h(n) = -(\text{unvisited edges at } n)$
 - b. IF PriorityQueue not empty:
 - Pop best neighbor
 - Move there, mark lane as visited
 - Append to path
 - c. ELSE (stuck - all adjacent lanes covered):
 - BFS to find nearest node with unvisited lanes
 - Walk backtrack path to that node
4. Write path to outputPS9.txt

6.3 Complexity Analysis

Metric	Theoretical	Actual (Measured)
--------	-------------	-------------------

Time	$O(E \log E)$	16.0000 ms
------	---------------	------------

Space	$O(V + E)$	2.2207 KB
-------	------------	-----------

- **Time:** Each edge is processed once; heap operations cost $O(\log E)$
 - **Space:** Adjacency list $O(V+E)$ + visited set $O(E)$ + path list $O(E)$
-

7. Sample Output

Starting Point: Marina Beach

Surveillance Path:

Marina Beach → Mahabalipuram → Vandalur Zoo → Kovalam Beach
→ Muttukadu → Kovalam Beach → Mahabalipuram → Government Museum
→ Marina Beach

Total Distance: 246 km | **Total Steps:** 9 | **Lanes Covered:** 7/7

8. Alternate Modelling Approach

Using Eulerian Path / Hierholzer's Algorithm

Description: The problem of covering all edges exactly once is equivalent to finding an **Eulerian Path** in graph theory. Hierholzer's Algorithm finds an Eulerian circuit in $O(E)$ time if one exists.

Conditions for Eulerian Path:

- All nodes must have even degree (for Eulerian circuit), OR
- Exactly two nodes have odd degree (for Eulerian path)

Performance Comparison:

Aspect	GBFS (Used)	Eulerian/Hierholzer
Time Complexity	$O(E \log E)$	$O(E)$
Space Complexity	$O(V + E)$	$O(V + E)$
Optimality	Heuristic (not always optimal)	Optimal (guaranteed)
Handles all graphs	Yes (with backtracking)	Only if Eulerian path exists
Implementation	More complex	Simpler

Why GBFS was chosen:

- Assignment specifically requires GBFS
- GBFS is more flexible — works even when a perfect Eulerian path doesn't exist
- Backtracking handles graphs where some lanes must be revisited
- Demonstrates AI search concepts (heuristic, priority queue)

Limitation of Hierholzer:

- Fails if the graph has more than 2 nodes with odd degree
- Cannot handle disconnected graphs without modification

9. Error Handling

Scenario	Handling
----------	----------

Input file not found	<code>FileNotFoundError</code> with clear message
Invalid file format	<code>ValueError</code> with expected format shown
Starting node not in graph	<code>ValueError</code> with valid options listed
Empty priority queue pop	<code>IndexError</code> with message
Duplicate edge added	Warning printed, edge skipped
Node not in graph	<code>ValueError</code> raised immediately

10. Conclusion

The City Surveillance Agent successfully uses **Greedy Best First Search** to cover all 7 lanes in Chennai's tourist map starting from Marina Beach. The heuristic function (unvisited edges count) effectively guides the drone toward areas with more remaining work. When the drone gets stuck, BFS backtracking ensures all lanes are eventually covered. The solution runs in **16 ms** using **2.2 KB** of memory, making it highly efficient for this problem scale.
